

Attela Beach Resort

Booking & Experience Platform

Full Product & Technical Specification

PRD · TRD · UI/UX · Information Architecture · Security · Database · Deployment ·
Implementation Plan

*Prepared for: Attela Beach Resort, Kisumu, Kenya
Version 1.0 — June 2026*

1. Executive Summary

Attela Beach Resort is a high-traffic lakeside venue on Lake Victoria in Kisumu, with over 41,000 combined followers across Facebook and Instagram, yet all bookings currently happen manually by phone or WhatsApp. This creates lost reservations, scheduling conflicts, and an experience that under-delivers relative to the brand's visual popularity.

This document specifies a booking and experience web application that lets visitors book tables, boat rides, event gardens, and staycations directly online, while giving resort staff full control over content and bookings through a built-in admin CMS — no developer required for day-to-day changes (photos, prices, events, availability).

The product is scoped to be buildable by a small two-person team using AI-assisted tools (v0 for UI generation, Supabase for backend/database) and deployable on a low-cost hosting stack appropriate for the resort's scale.

2. Product Requirements Document (PRD)

2.1 Problem Statement

Customers discover Attela through high-quality video content but hit friction the moment they want to act: they must call or WhatsApp manually, with no visibility into availability, no self-service confirmation, and no single verified source of truth — a gap that has already been exploited by a fraudulent event poster impersonating the brand.

2.2 Goals & Success Metrics

Goal	Metric
Reduce manual booking load on staff phones	% of bookings made via app vs. phone within 90 days of launch
Increase conversion from social media traffic	Click-through rate from Instagram/Facebook bio link to booking completion
Eliminate impersonation/scam confusion	Single verified domain referenced in all official social posts
Give staff independence from developers	% of content updates (photos, prices, events) made without external help

2.3 Scope

In scope (v1):

- Public-facing marketing site (hero, experiences, gallery, menu, events, contact)
- Online booking flow for Table, Boat Ride, Event Garden, Staycation
- Admin CMS for managing all content and bookings without code changes
- WhatsApp-relay or direct booking confirmation flow
- Mobile-first responsive design

Out of scope (v1 — future phases):

- Online payment processing (M-Pesa integration deferred to v2)
- Multi-language support
- Loyalty/rewards program

- Native mobile app (web app only for v1)

3. User Types & Personas

User Type	Description	Key Needs
Guest / Visitor	Anyone browsing the site to learn about or book an experience	Fast load, clear pricing, simple booking, trust signals (real photos, verified contact)
Returning Customer	Has booked before, may look for past bookings or repeat events	Quick rebooking, awareness of upcoming events
Front Desk / Booking Staff	Resort staff confirming and managing day-to-day bookings	Simple dashboard to view, confirm, or adjust bookings without technical training
Resort Admin / Manager	Owner or manager managing the overall content and offerings	Full CMS control: edit photos, prices, events, experiences, and view booking analytics
Super Admin (You / Developer)	Technical maintainer of the platform	Access to system settings, integrations, backups, and user permission management

4. Information Architecture

4.1 Public Site Map

- Home (Hero, Signature Experiences, Booking CTA, Gallery preview, Events preview)
- Experiences (Sunset Boat Cruise / DJ Nights / Floating Bridge / Family Garden Staycation — each with its own detail view)
- Book Now (category tabs: Chill & Dine, Party, Family Day, Boat Ride → booking form)
- Gallery (photo/video grid)
- Menu & Pricing
- Events Calendar
- Contact (WhatsApp, phone numbers, Instagram, map, verified-contact notice)

4.2 Admin CMS Map

- Dashboard (today's bookings, quick stats, alerts for new requests)
- Bookings Manager (list, filter by date/category/status, confirm/cancel/reschedule)
- Content Manager → Hero (video/image, tagline)
- Content Manager → Experiences (add/edit/remove experience cards)
- Content Manager → Gallery (upload/remove photos & videos)
- Content Manager → Menu & Pricing (edit items, prices, categories)
- Content Manager → Events Calendar (add/edit/remove upcoming events)
- Settings (contact numbers, social links, opening hours, map location)
- Users & Roles (manage staff accounts and permission levels)

5. CMS & Admin Editability Requirements

Every piece of customer-facing content must be editable by a non-technical admin through a secured dashboard, with changes reflected live on the public site without a redeploy.

Editable Element	Who Can Edit	Notes
Hero video/image & tagline	Admin	Upload replaces current asset; preview before publish
Experience cards (title, photo, description)	Admin	Add, edit, reorder, or remove cards
Gallery photos/videos	Admin, Staff	Bulk upload supported; staff can upload, only admin can delete
Menu items & prices	Admin	Organized by category (Food / Drinks / Boat Rides)
Events calendar entries	Admin, Staff	Title, date, description, optional poster image
Booking availability/blackout dates	Admin	Block specific dates/times per category
Contact info (numbers, social links, hours)	Admin	Single source of truth shown across the site
Booking statuses	Admin, Staff	Confirm, reschedule, cancel, mark as completed
Staff accounts & permissions	Super Admin	Add/remove staff logins, assign role

Implementation approach: all editable content is stored in the database (not hardcoded), rendered dynamically on the public site, and exposed through a permissioned admin UI built on the same stack (Next.js + Supabase) so no separate CMS license or service is required.

6. UI/UX Design Requirements

6.1 Visual Direction

- Palette: warm sunset tones (deep orange, gold) paired with deep navy/teal accents
- Typography: large, highly readable headings; clean sans-serif body text
- Imagery-first: hero video, large photo cards — the brand's existing content is the main asset
- Rounded, card-based components throughout; generous spacing; tropical, energetic feel
- Avoid corporate/sterile patterns — this is a lifestyle brand, not a formal hotel chain

6.2 Core UX Principles

- Mobile-first: majority of visitors will arrive from Instagram/Facebook on phones
- Persistent “Book Now” access from any scroll position
- Booking flow completable in under 60 seconds, 4 fields or fewer beyond category selection
- Single verified contact path repeated in header and footer to counter impersonation confusion
- Admin dashboard prioritizes clarity over density — large buttons, minimal jargon, confirmation prompts before destructive actions (delete, cancel booking)

6.3 Key Screens

- Public: Home, Experience Detail, Booking Flow (category → form → confirmation), Gallery, Menu, Events, Contact
- Admin: Login, Dashboard, Bookings List/Detail, Content Editors (per section above), Settings, User Management

7. Technical Requirements Document (TRD)

7.1 Recommended Stack

Layer	Technology	Reason
Frontend framework	Next.js (React)	Pairs directly with v0-generated components; strong SEO support via server rendering
UI generation	v0 by Vercel	Rapid generation of styled components matching the brand direction
Styling	Tailwind CSS	Default in v0 output; fast iteration, consistent design tokens
Backend/Database	Supabase (Postgres)	Managed Postgres + built-in auth + storage + auto-generated APIs; low ops overhead for a 2-person team
Authentication	Supabase Auth	Email/password login for admin and staff roles, role-based access control
File/Media storage	Supabase Storage	Hosts gallery photos/videos and hero media with CDN delivery

Hosting	Vercel	Native Next.js deployment, automatic HTTPS, global CDN, generous free tier for this scale
Notifications	WhatsApp Business API or wa.me relay	Booking confirmations routed to resort's verified WhatsApp number

7.2 Functional Requirements Summary

- FR1: Visitors can view all public content without authentication
- FR2: Visitors can submit a booking request specifying category, date, time, party size, and contact info
- FR3: System sends a confirmation notification (WhatsApp/email) to both the resort and the requester
- FR4: Authenticated staff can view, filter, and update booking status
- FR5: Authenticated admins can create, edit, reorder, and delete all content entities (experiences, gallery items, menu items, events)
- FR6: Super admin can manage staff accounts and permission levels
- FR7: All content changes reflect on the public site immediately without a code deploy

8. Database Design

Relational schema on Postgres (via Supabase). Core entities below; all tables include standard id (uuid), created_at, and updated_at fields.

Table	Key Fields	Purpose
users	id, email, role (admin/staff/super_admin), full_name	Authentication & role-based access
bookings	id, category, date, time, party_size, customer_name, phone, status, notes	Stores all booking requests and their lifecycle
experiences	id, title, description, image_url, sort_order, is_active	Signature experience cards shown on the homepage
gallery_items	id, media_url, media_type, caption, sort_order	Photo/video gallery content
menu_items	id, category, name, description, price, currency, is_available	Menu/pricing content
events	id, title, description, event_date, poster_url, is_active	Events calendar entries
availability_blocks	id, category, blocked_date, blocked_time_range, reason	Admin-defined blackout dates/times
site_settings	id, key, value	Single key-value table for contact info, hours, social links, map coordinates
audit_log	id, user_id, action, entity, entity_id, timestamp	Tracks who changed what, for accountability

Booking status lifecycle: pending → confirmed → completed or pending → cancelled. Status transitions are restricted to staff/admin roles.

9. Security Requirements

- Role-based access control (RBAC): Guest (no auth), Staff, Admin, Super Admin — enforced at the database level via Supabase Row Level Security (RLS) policies, not just in the UI
- All admin/staff routes require authentication; sessions expire after a defined inactivity period
- Passwords hashed and managed entirely by Supabase Auth; no custom password storage
- Input validation and sanitization on all booking form submissions to prevent injection attacks
- Rate limiting on the public booking endpoint to prevent spam/abuse
- HTTPS enforced everywhere via the hosting provider's default TLS
- Audit log of all content and booking changes for accountability and dispute resolution
- Media uploads restricted by file type and size; stored in access-controlled buckets
- Environment secrets (API keys, database credentials) stored in hosting provider's secret manager, never committed to source control

10. Performance Requirements

Requirement	Target
First page load (mobile, 3G/4G)	Under 3 seconds for initial meaningful content
Booking form submission response	Under 1.5 seconds
Image/video delivery	Served via CDN with automatic compression and responsive sizing
Admin dashboard load	Under 2 seconds for booking list of up to 500 records
Uptime target	99.5% (appropriate for a single-location SMB; not requiring enterprise SLAs)

11. Deployment Architecture

A simple, low-maintenance architecture suited to a two-person team with no dedicated DevOps resource.

- Frontend (Next.js app) deployed on Vercel, connected directly to the project's Git repository for automatic deployment on every push to the main branch
- Backend services (database, auth, storage, auto-generated REST/Realtime APIs) hosted on Supabase
- Custom domain (e.g. attelabeachresort.com) pointed to Vercel via DNS, with automatic SSL
- Separate staging and production environments/branches to test changes before they go live
- Automated daily database backups via Supabase's built-in backup feature
- Media assets served through Supabase Storage's CDN layer

This architecture requires no servers to manage directly — both Vercel and Supabase are managed platforms, which keeps ongoing maintenance load low and appropriate for the team size.

12. Implementation Plan

Phase	Deliverables
1. Foundation	Supabase project setup, database schema created, Next.js project scaffolded, v0-generated UI components imported
2. Public Site Build	Home, Experiences, Gallery, Menu, Events, Contact pages built and wired to live database content
3. Booking Flow	Booking form, submission handling, WhatsApp/email confirmation notifications, availability blocking logic
4. Admin CMS	Authentication, role-based dashboard, content editors for every entity in Section 5, booking management screen
5. Security & QA Hardening	RLS policies, input validation, rate limiting, cross-device testing, content load testing
6. Content Population & Launch	Real photos/videos and pricing loaded by resort staff, soft launch, monitor first real bookings, iterate based on staff feedback

13. Final Development Instruction

Use this as the single instruction block to hand to a build tool (v0, Claude Code, or a developer) to begin implementation.

Build a full-stack booking and content management web application for Attela Beach Resort, a lakeside restaurant and event venue in Kisumu, Kenya.

Stack: Next.js (React) frontend with Tailwind CSS, Supabase for database/auth/storage, deployed on Vercel.

Public site: Build Home (video hero, signature experience cards, booking CTA), an Experiences page, a Booking flow with category tabs (Chill & Dine, Party, Family Day, Boat Ride) and a short form (date, time, party size, name, phone), a Gallery, a Menu/Pricing page, an Events Calendar, and a Contact section with WhatsApp click-to-chat, both phone numbers, Instagram link, and an embedded map. Use a warm sunset color palette with navy/teal accents, rounded card UI, and mobile-first responsive layout throughout.

Admin CMS: Build an authenticated dashboard (role-based: staff, admin, super admin) where authorized users can edit every public-facing content entity — hero media, experience cards, gallery items, menu items, events, contact/site settings — and manage bookings (view, filter, confirm, reschedule, cancel) without needing code changes or redeployment.

Database: Implement the schema in Section 8 (users, bookings, experiences, gallery_items, menu_items, events, availability_blocks, site_settings, audit_log) using Supabase Postgres with Row Level Security policies enforcing the role permissions defined in Section 5 and Section 9.

Non-negotiables: mobile-first performance (sub-3-second load), booking confirmation notification on every submission, single verified-contact messaging in the footer, and zero hardcoded content — everything editable by an admin must be pulled from the database.

Build in the phased order set out in Section 12, starting with database schema and the public site before layering in the admin CMS and security hardening.

This specification is intentionally scoped for a small two-person build team using AI-assisted tooling. Treat Section 12's phases as checkpoints — validate each with the resort before moving to the next.